TFR/911H NitrOS-9 Reference Pages

Commands, Special Files, Games, and Maintenance

Order Number: TFR-911H-RP-A

This manual describes the commands, games, and tools added to NitrOS-9 Level 2 on the TFR/911H single board computer, the hardware the TFR/911H firmware presents to OS-9, and how to build and maintain the system. It begins with an introduction to OS-9 Level 1 and Level 2 as they run on the board.

Revision/Update Information:	This is a new manual.
Operating System and Version:	NitrOS-9/6809 Level 2 V3.3.0
Hardware and Firmware:	TFR/911H, firmware v4

the tfr/9 project

github.com/strickyak/tfr9

First Printing, October 2026

The information in this document is subject to change without notice and should not be construed as a commitment by anyone in particular. The software described in this document is furnished as source code, and may be used, copied, and changed under the terms of the licenses that come with it.

No responsibility is assumed for the use or reliability of software on equipment that was soldered together at a kitchen table.

© The TFR/9 Project contributors, 2026. All rights reserved.

Trademarks. OS-9 is a trademark of Microware Systems Corporation. NitrOS-9 is the work of the NitrOS-9 Project. Zork is a registered trademark of Infocom, Inc. Tetris is a trademark of The Tetris Company. Raspberry Pi is a trademark of Raspberry Pi Ltd. Color Computer and CoCo are associated with Tandy Corporation. ULTRIX and DEC are trademarks of Digital Equipment Corporation.

About the design. This manual is typeset in the manner of Digital Equipment Corporation's ULTRIX reference pages of the early 1990s, as a tribute. It is not a Digital document, and it has nothing to do with Digital Equipment Corporation or its successors.

This document was prepared with LaTeX from `v4/nitros9-l2/doc/manual.tex`.

Contents

About This Manual iv

1 Introduction 1–1

1.1 The TFR/911H . . . 1–1
1.2 OS-9 Level 1 and Level 2 . . . 1–1
1.3 Level 1 on the TFR/911H: TurbOS . . . 1–2
1.4 Level 2 on the TFR/911H: NitrOS-9 V3.3.0 . . . 1–2
1.4.1 The machine NitrOS-9 sees . . . 1–2
1.4.2 The disk . . . 1–2
1.4.3 The modules . . . 1–3
1.5 What Was Added . . . 1–3
1.6 A First Session . . . 1–3

2 Reference Pages 2–1

Section 1: Commands . . . 2–2
beep(1) *ring the bell* . . . 2–3
clear(1) *clear the screen* . . . 2–4
click(1) *turn keyclicks on or off* . . . 2–5
grep(1) *print lines that contain a string* . . . 2–6
info(1) *show the system and the hardware* . . . 2–7
pwd(1) *print the data or execution directory* . . . 2–9
unix(1) *Unix-style names for OS-9 commands* . . . 2–10
vi(1) *a small vi* . . . 2–11
Section 4: Special Files . . . 2–13
sn76489(4) *the sound chip* . . . 2–14
tfr911h(4) *the hardware the firmware emulates* . . . 2–15
Section 6: Games . . . 2–16
tetris(6) *Tetris, with music* . . . 2–17
zm(6) *a Z-machine, for Infocom's games* . . . 2–18
Section 8: Maintenance . . . 2–19
nitros9-l2(8) *build the system and keep a disk up to date* . . . 2–20
tcl(8) *the firmware's console* . . . 2–21
tether(8) *the board's terminal, disk, and clock* . . . 2–22

A Connecting a Speaker A–1

A.1 Wiring . . . A–1

A.2 Trying It A–1
A.3 How It Works A–1

B How the Software Was Tested B–1

B.1 zm B–1
B.2 vi B–1
B.3 info B–1
B.4 The Firmware B–1

About This Manual

This manual describes the software that turns a TFR/911H into a small, complete OS-9 system: NitrOS-9 Level 2, booted by the board's firmware, with a disk, a clock, a Unix-flavored set of commands, an editor, a sound chip, and games. It also describes the hardware that the firmware emulates for OS-9, and how to rebuild everything.

Audience

This manual is for anyone with a TFR/911H on their desk: people who know some Unix and want to find their way around OS-9, CoCo veterans who want to know what is different about this machine, and people who want to change the firmware or the disk. No knowledge of OS-9 is assumed in Chapter 1.

Organization

This manual has two chapters and two appendices:

Chapter 1 Introduces the TFR/911H, OS-9 Level 1 and Level 2, the TurbOS and NitrOS-9 systems that run on the board, and a first session.

Chapter 2 Contains the reference pages, grouped as ULTRIX grouped them: Section 1, Commands; Section 4, Special Files (the hardware); Section 6, Games; and Section 8, Maintenance.

Appendix A Explains how to connect a speaker.

Appendix B Describes how the software was tested.

Related Documents

- `v4/nitros9-l2/README.md`, which this manual expands on.
- The NitrOS-9 documentation, at `nitros9.org` and in the NitrOS-9 source tree under `docs`.
- *OS-9 Level Two* for the Tandy Color Computer 3, whose commands NitrOS-9 still follows closely.
- The TFR/9 hardware reference, on the `strickyak/tfr9` wiki.

Conventions

Convention	Meaning
`OS9:`	The NitrOS-9 Shell prompt.
`TCL>`	The TFR/911H firmware's console prompt, before an OS boots.
`$`	A shell prompt on the computer the board is plugged into.
`bold type`	In examples, what you type.
italic	Something you supply: a file name, a number.
`[ ]`	In syntax, an optional part. Do not type the brackets.
|	In syntax, a choice: `on`|`off` means `on` or `off`.
Ctrl/C	Hold down the Ctrl key while you press C. Also written `^C`.
Return	The Return (Enter) key.
cmd(n)	The reference page for **cmd** in Section *n*.
`$FF41`	A hexadecimal number, often an address in the 6309's I/O page.

Introduction 1

This chapter introduces the TFR/911H, the two levels of OS-9, the two OS-9 systems the board runs, and what was added to NitrOS-9 Level 2 for it. It ends with a first session at the keyboard.

1.1 The TFR/911H

The TFR/911H is a single board computer built around a genuine Hitachi HD6309 microprocessor—the CMOS cousin of Motorola's 6809, with extra registers and a faster native mode—and a Raspberry Pi RP2350B microcontroller. Everything that is not the CPU is done by the RP2350B:

- It makes the 6309's E and Q clocks, one bus cycle at a time.
- It is the memory: 256K of the RP2350B's 520K of SRAM, behind an emulated MMU.
- It is every I/O device: a serial port for the console, a hard disk, a clock, interrupt timers, a sound chip.

One of the RP2350B's two cores runs the 6309's bus; the other talks USB to a program on your computer, the *tether*, which is the board's terminal, its disk drive, and its clock. The 6309 runs at about 2.8 to 3.2 MHz—faster than the 1.79 MHz of the fastest Color Computer.

The TFR/9 family of boards was designed to make very cheap 6809-family computers that run OS-9. Its design files and software are at `github.com/strickyak/tfr9`.

1.2 OS-9 Level 1 and Level 2

OS-9 was written by Microware Systems Corporation in 1979 and 1980 for the Motorola 6809. It is a real-time, multitasking, multi-user operating system that will look familiar to anyone who knows Unix: a hierarchical file system, a shell with pipes and redirection, devices that are files, processes with parents and children. It was written to be small and to run from ROM, so everything in OS-9 is a *module*: a position-independent, re-entrant block of code or data with a header and a CRC, which the system can find in memory by name. The kernel, the device drivers, and the commands you run are all modules.

The 6809 can address only 64K of memory, and OS-9 comes in two *levels* that differ in what they do about that.

Level 1 runs everything in a single 64K address space: the kernel, every process, every module. It needs no memory management hardware, which made it right for the first Color Computers. Tandy sold OS-9 Level One for the Color Computer in 1983.

Level 2 uses a memory management unit to give each process its own 64K address space, made of 8K blocks drawn from a larger physical memory—up to 2M. The kernel lives in a space of its own. Processes cannot see or damage each other's memory, and many more can run at once. Tandy sold OS-9 Level Two for the Color Computer 3, whose GIME chip has the MMU, in 1987.

NitrOS-9 began in the 1990s as a rewrite of OS-9 that made use of the Hitachi 6309's extra registers and instructions. Today the NitrOS-9 Project maintains it as open source, for both the 6309 and the 6809, for Level 1 and Level 2, and for many machines besides the Color Computer.

1.3 Level 1 on the TFR/911H: TurbOS

When the TFR/911H firmware starts, it waits at its console prompt, `TCL>`, a small Tcl interpreter (see **tcl**(8)). Typing **`bye`** there starts *TurbOS*, a small OS-9 Level 1 system that the firmware loads into the 6309's memory as a single 64K image, from its own flash.

TurbOS runs without a disk. Its kernel, I/O manager, SCF terminal manager, terminal driver, Shell, and a handful of commands (**mdir**, **mfree**, **procs**, **sleep**) are modules in the image. Its prompt is `TOS:`:

```
TOS:mdir
 Module directory at 00:00:59
kernel  init  tk  ioman  scf  scvt  term  go  shell  mfree  mdir  procs  sleep
```

TurbOS comes from the TFR/9 project's TurboLab work, in the `v4/turbolab` directory of the TFR/9 source. The rest of this manual is about Level 2.

1.4 Level 2 on the TFR/911H: NitrOS-9 V3.3.0

Typing **`l2`** at the `TCL>` prompt starts NitrOS-9/6809 Level 2 V3.3.0, built for the Color Computer 3. The firmware copies a small launcher and the kernel's boot track into memory and resets the 6309, which boots in about half a second:

```
NitrOS-9/6809 Level 2 V3.3.0
Tandy Color Computer 3
(C) 2014 The NitrOS-9 Project
Fri Oct 16 17:23:34 2026
http://www.nitros9.org

NitrOS-9 Level 2 on the TFR/911H.  Unix-style commands: man unix

Shell

OS9:
```

1.4.1 The machine NitrOS-9 sees

NitrOS-9 thinks it is on a Color Computer 3. The firmware emulates as much of one as OS-9 uses, and a little more:

Device	Address	What it is
RAM		256K in 32 blocks of 8K; 184K free for programs
GIME MMU	`$FF90`–`$FF91`, `$FFA0`–`$FFAF`	Two task maps; the constant page at `$FE00`
GIME timer	`$FF92`	The VBORD interrupt, 60 times a second: the system clock
6850 ACIA	`$FF14`–`$FF15`	The console, `/Term`
SN76489	`$FF41`	A sound chip: three tones and noise
Control	`$FF42`	Keyclicks, the bell, raw arrow keys
Info port	`$FF43`–`$FF44`	The firmware's report on itself
EmuDsk	`$FF80`–`$FF86`	A virtual hard disk, `/DD` and `/H0`
Clock	`$FFC0`–`$FFC7`	Date and time, from your computer

tfr911h(4) describes each of these.

1.4.2 The disk

The disk is a file on your computer, `v4/nitros9-l2/level2.dsk`: an RBF disk of 99,999 sectors (24.4M) that the tether serves to EmuDsk a sector at a time. It holds `OS9Boot`, the rest of the operating system, which the kernel loads at boot; the commands in `/DD/CMDS`; the system files `errmsg` and `helpmsg` in `/DD/SYS`; and story files for **zm**(6) in `/DD/GAMES`. Whatever you save goes to the file at once.

1.4.3 The modules

These are in memory once the system is up (**info**(1) lists them):

Modules	What they do
`REL Boot Krn`	Relocate the boot track, load `OS9Boot` from EmuDsk, and the kernel
`Init`	The system's settings: startup program, devices, table sizes
`KrnP2 KrnP3`	The rest of the kernel, and its error-message extension
`IOMan`	The I/O manager
`Clock Clock2`	The 60 Hz tick, and the real-time clock (`clock2_tfr911`)
`SCF sc6850 Term`	The terminal file manager, the 6850 driver, `/Term`
`RBF EmuDsk DD H0`	The disk file manager, the EmuDsk driver, `/DD` and `/H0`
`PipeMan Piper Pipe`	Pipes
`SysGo Shell`	The first process, and the Shell it starts

1.5 What Was Added

These are the commands, games, and tools described in Chapter 2. Each has a **man** page on the board too: type **`man`** *name*.

Reference page	Summary
unix(1)	**ls**, **cd**, **cat**, **cp**, **mv**, **rm**, **ps**, **df**, and other Unix names for OS-9 commands
vi(1)	A small vi, also called **vim**
info(1)	Everything the system knows about itself
grep(1)	Print lines that contain a string
pwd(1)	Print the data or execution directory
clear(1)	Clear the screen
beep(1), **click**(1)	Ring the bell; turn keyclicks on and off
tfr911h(4)	The hardware the firmware emulates
sn76489(4)	Programming the sound chip
tetris(6)	Tetris, with music
zm(6)	A Z-machine, for Zork and the other Infocom games
tether(8)	The program that is the board's terminal and disk
tcl(8)	The firmware's console: **l2**, **bye**, **beep**, **click**
nitros9-l2(8)	Building the system and keeping a disk up to date

1.6 A First Session

Plug the TFR/911H into a USB port and, in the `v4` directory of the TFR/9 source, run `run-l2.sh`. It starts the tether, which boots NitrOS-9 by typing **`l2`** for you (or reconnects, if NitrOS-9 is already running):

```
$ WIRE=/dev/cu.usbmodem31201 sh run-l2.sh
TFR/911H on /dev/cu.usbmodem31201, disk .../nitros9-l2/level2.dsk.
^C goes to OS-9; quit with ^] .
```

Then look around. **man unix** lists the Unix-style names; **ls -l** lists a directory; **info** shows the whole system:

```
OS9:ls -l /dd/cmds
OS9:cd /dd
OS9:pwd
/DD
OS9:info
```

Write a file with **vi**, and look at it with **cat**:

```
OS9:vi notes
OS9:cat notes
```

Play something:

```
OS9:zm zork1
OS9:tetris
```

Ctrl/C interrupts the program that is running, as on Unix. To leave the tether, press Ctrl/]; NitrOS-9 keeps running on the board, and `run-l2.sh` will reconnect to it.

OS-9 differs from Unix in a few ways you will meet early:

- Each process has two current directories: the *data* directory, for files (**cd**, **pwd**), and the *execution* directory, where commands are found (**chx**, **pxd**). Both start at `/DD` and `/DD/CMDS`.
- The pipe symbol is `!`, not `|`. `>>` redirects standard error; there is no append. `&` runs a command in the background, as on Unix.
- File names are not case sensitive. Text files end their lines with a carriage return, not a line feed.

Reference Pages 2

The reference pages follow, in four sections. Within a section they are in alphabetical order. Each page has the same parts, where they apply: **Name**, **Syntax**, **Description**, **Options**, **Restrictions**, **Examples**, **Files**, and **See Also**. The commands that came with NitrOS-9 are not described here; use **man** on the board for those.

Section 1: Commands

This section describes the commands added to NitrOS-9 Level 2 for the TFR/911H. The commands that come with NitrOS-9 are described by **man** on the board, and in the NitrOS-9 documentation.

Name	Description
beep(1)	Ring the bell
clear(1)	Clear the screen
click(1)	Turn keyclicks on or off
grep(1)	Print lines that contain a string
info(1)	Show what the system and the hardware know about themselves
pwd(1)	Print the data or execution directory
unix(1)	Unix-style names for OS-9 commands: ls, cd, cat, cp, mv, rm, ps, df, …
vi(1)	A small vi, also called vim

Name **beep** – ring the bell

Syntax **beep**

Description The **beep** command writes a bell character (`$07`) to standard output. The terminal the tether runs in rings its bell, and the TFR/911H firmware, which sees the character go out through the 6850, plays an 880 Hz tone for 150 ms through the board's speaker, if one is connected and bit 1 of the control register at `$FF42` is set (it is, at power on).

Any program that prints `^G` makes the same sound.

Examples To sound the bell when a long command finishes:

```
OS9:copy /dd/bigfile /dd/backup; beep
```

See Also **click**(1), **sn76489**(4), **tfr911h**(4)

Name **clear** – clear the screen

Syntax **clear**

Description The **clear** command sends the ANSI sequences that move the cursor home and clear the screen of the terminal the tether runs in.

See Also **unix**(1)

Name **click** – turn keyclicks on or off

Syntax **click** `on|off`

Description When keyclicks are on, the TFR/911H's speaker gives a short tick each time a key reaches OS-9. The **click** command turns them on or off by setting or clearing bit 0 of the control register at `$FF42`. They are on at power on. With no argument, or any other, **click** prints its usage.

The `TCL>` console has a **click** command of its own, which does the same before an OS is booted; see **tcl**(8).

Restrictions **tetris**(6) turns keyclicks off while it runs, and puts them back as they were when it ends.

See Also **beep**(1), **sn76489**(4), **tfr911h**(4)

Name **grep** – print lines that contain a string

Syntax **grep** [`-i`] [`-n`] *pattern* [*path*]

Description The **grep** command prints each line of *path*, or of standard input if no path is given, that contains *pattern*. The pattern is a plain string, not a regular expression. It may be given in double quotes, which it needs if it contains spaces, or without them.

This is NitrOS-9's **grep**, changed in two ways: the pattern need not be quoted, and it ends cleanly at the end of a file, where the original reported a spurious error.

Options

`-i`	Ignore case.
`-n`	Put each line's number in front of it.

Examples

```
OS9:grep -n total sample.c
OS9:dir -e ! grep -i tetris
```

See Also **unix**(1)

Name

info – show what the system and the hardware know about themselves

Syntax

info [-p]

Description

The **info** command prints a report in eight parts:

System	The OS name and version, the machine, the kernel's level and version and the CPU it was built for, the startup program, disk and console, the sizes of the device and interrupt tables, and the date. These come from the `Init` module.
Memory	The RAM, what is free (and the largest piece that is free), and what is used by modules and by everything else, from the system's memory block map.
Processes	Every process: its number, its parent, its user, its priority, its memory, its state, and the program it runs. The one marked `*` is **info** itself.
Devices	Every device descriptor in memory, with its file manager, driver, port, and drive, and how many paths are open on it. A device that nothing is using is not attached, and shows `-`.
Disks	Each disk drive (once, though several descriptors may name it): its name, the date it was made, its size, and its free space, from its sector 0 and its allocation bitmap.
Interrupts	The entries in the IRQ polling table, with the driver and device each belongs to.
Modules	The names of the modules in memory, and their total size.
Hardware	The TFR/911H firmware's own report: the board, the firmware's build date, the RP2350B's clock, memory and ID, the 6309's measured speed and how busy it is, the interrupt rate, the disk, console and sound counters, whether the clock was set, and the uptime.

The kernel's tables are in the system's own address space. **info** reads them as NitrOS-9's **procs**, **devs**, and **mdir** do: it gets the system process's descriptor with `F$GPrDsc`, and uses the memory map in it to copy what it needs with `F$CpyMem`. The hardware report comes from the firmware's info port; see **tfr911h**(4).

Options

`-p`	Plain text: leave out the escape sequences that make the headings bold, for saving the report to a file.

Restrictions

The RP2350B's temperature sensor reads nonsense on some boards, so the chip's temperature is shown only when its reading is believable.

On firmware without the info port, the hardware part says so and is otherwise empty.

Examples

Part of a report, taken with **tetris**(6) running in the background:

```
OS9:info
NitrOS-9/6809 Level 2 V03.3.0 on the TFR/911H

System
  OS          NitrOS-9/6809 Level 2 V03.3.0
  Machine     Tandy Color Computer 3, as the TFR/911H pretends to be
  Kernel      Level 2, version 3.3.0, built for the 6809
  Startup     runs SysGo, disk /DD, console /Term
  Tables      39 device slots, 21 IRQ polling slots
  Date        2026/10/17 01:25:15

Memory
  RAM         248K in 31 blocks of 8K
  Free        160K (largest piece 152K)
  In use      24K modules, 64K system and processes

Processes
   ID  Parent  User  Prio  Memory  State      Program
    1     0       0     255   0                 (the system)
    2     1       0     128   768     waiting   Shell
    3     2       0     128   4K      waiting   tetris
   *4     2       0     128   6K      running   info
```

```
Devices
  Device    Type  Manager  Driver    Port   Drive  Users
  /Term       SCF   SCF      sc6850    $FF14         4
  /DD         RBF   RBF      EmuDsk    $FF80  0      5
  /H0         RBF   RBF      EmuDsk    $FF80  0      -
  /Pipe       Pipe  PipeMan  Piper     $0000         -

Disks
  /DD = /H0
              "tfr911-level2.dsk", made 2026/10/17
              24.4M (99999 sectors), 24.0M free (98%)

Hardware (from the TFR/911H firmware)
  Board       TFR/911H: a Hitachi 6309 run by a Raspberry Pi RP2350B
  Pico        RP2350B at 250 MHz, 520K SRAM
  6309        3.24 MHz (measured), 70% busy, in 6809 mode
  Interrupts  59 IRQs/s: GIME VBORD 60 Hz clock, 6850 ACIA
  Sound       SN76489 at $FF41 (182 writes), PWM audio on GPIO 8, keyclick off
  Uptime      Pico 0m 56s, NitrOS-9 0m 54s
```

To keep a report:

```
OS9:info -p >/dd/sysinfo
```

See Also **tfr911h**(4); NitrOS-9's **mdir**, **mfree**, **procs**, **devs**, **irqs**, **free**

Name **pwd**, **pxd** – print the data or execution directory

Syntax
pwd
pxd

Description An OS-9 process has two current directories. The **pwd** command prints the *data* directory, where file names that do not begin with / are looked for, and which **cd** (or **chd**) changes. The **pxd** command prints the *execution* directory, where commands are looked for, which **chx** changes.

Both are NitrOS-9's **pd**, built twice.

Examples
```
OS9:cd /dd/games
OS9:pwd
/DD/GAMES
OS9:pxd
/DD/CMDS
```

See Also **unix**(1)

Name

unix – Unix-style names for OS-9 commands

Syntax

ls [`-l`] [*path*] **cd** *path* **cat** *path* …

Description

The TFR/911H's NitrOS-9 knows the names a Unix user reaches for first. Most are *aliases*: tiny modules that run the OS-9 command that does the job, passing along their arguments. Two are built into the Shell, and the rest are real commands. **`man unix`** on the board lists them.

Name	Runs	What it does
ls [`-l`]	**dir** [`-e`]	List a directory; `-l` gives the long form
ll	**dir -e**	List a directory, long form
cat	**list**	Print a text file
cp	**copy**	Copy a file
mv	**rename**	Rename a file or directory
rm	**del**	Delete files
mkdir	**makdir**	Make a directory
rmdir	**deldir**	Delete a directory and what is in it
ps	**procs**	List processes; `-e` lists everyone's
df	**free**	Show free space on a disk
less	**more**	Show a file a screen at a time
man	**help**	Show help for a command
uname	**echo**	Say what this is
vim	**vi**	The editor; see **vi**(1)
cd	built in	Change the data directory (the same as **chd**)
exit	built in	Leave this Shell
pwd, **pxd**	**pd**	See **pwd**(1)
grep, **clear**		See **grep**(1), **clear**(1)

An alias puts any extra arguments of its own in front of yours, and then chains to the real command with `F$Chain`, so it costs no extra process. **cd** and **exit** are added to the table of built-in commands in a copy of the Shell's source.

Restrictions

These are OS-9 commands under Unix names, and they behave as OS-9 commands. **mv** cannot move a file to another directory. **rmdir** deletes the directory's contents too, after asking. **rm** takes no options.

The Shell's syntax is OS-9's, not the Bourne shell's: the pipe symbol is `!`, `>>` redirects standard error (there is no append), `<>` and `>>>` redirect several paths at once, and `&` runs a command in the background.

Examples

```
OS9:ls -l /dd
OS9:mkdir /dd/src
OS9:cp notes /dd/src/notes
OS9:cd /dd/src
OS9:ps
```

Files

`nitros9-l2/unix.tcl`	The list of aliases. Add a line to it and rebuild to make another; see **nitros9-l2**(8).

See Also

grep(1), **pwd**(1), **vi**(1)

Name

vi, **vim** – a small vi

Syntax

vi [`-w` *cols*] [`-h` *rows*] [*file*]

Description

The **vi** command is a screen editor in the manner of the Unix **vi**, with most of the commands people use without thinking about them. It starts in *normal* mode, where keys are commands. The insert commands enter *insert* mode, where keys are text, until you press Esc. A colon starts a *command line* at the bottom of the screen.

The bottom line shows messages, `-- INSERT --` in insert mode, and the cursor's line and column. Lines longer than the screen are not wrapped: the view scrolls sideways.

Most normal-mode commands take a count in front: **`3dd`** deletes three lines, **`5j`** moves down five, **`2d3w`** deletes six words.

Moving

`h j k l`	Left, down, up, right. The arrow keys too.
`w b e`	To the next word, back a word, to the end of the word. `W B E` do the same with words that are anything but blanks.
`0 ^ $`	To the start of the line, its first non-blank, its end.
`G`, `gg`	To the last line, or to line *n* with a count; to the first line.
`H M L`	To the top, middle, or bottom of the screen.
`f`*c* `F`*c* `t`*c* `T`*c*	To the next (`f`) or previous (`F`) *c* on the line, or just before it (`t`, `T`). `;` repeats; `,` repeats the other way.
`%`	To the bracket that matches the next `( [ { } ] )`.
`{ }`	Back or on a paragraph (to a blank line).
`+ - Return`	Down or up a line, to its first non-blank.
`^F ^B`	Forward or back a screen. Page Down and Page Up too.
`^D ^U`	Down or up half a screen.

Searching

`/`*text*, `?`*text*	Find *text* forward or backward, wrapping around the file.
`n N`	Find it again, the same way or the other.
`*`	Find the word under the cursor.

Inserting

`i a`	Insert before, or append after, the cursor.
`I A`	Insert at the line's first non-blank, or append at its end.
`o O`	Open a new line below, or above.

In insert mode, Backspace deletes the character before the cursor (and can join lines), `^W` the word before it, `^U` the line before it; Return splits the line, Tab inserts a tab, and the arrow keys move the cursor. Esc goes back to normal mode.

Changing

`x X`	Delete the character under, or before, the cursor.
`D C`	Delete, or change, to the end of the line.
`s S`	Change a character, or the whole line.
`r`*c*	Replace the character with *c* (`r` Return splits the line).
`J`	Join this line and the next.
`~`	Switch the case of the character, and move on.
`>> <<`	Indent the line by a tab, or take one away.
`p P`	Put the last deleted or yanked text after, or before, the cursor (below or above, for whole lines).
`u`	Undo the last change. Again and again, back to the start.
`.`	Do the last change again.

Operators d (delete), c (change), y (yank), > and < (indent) work on the text that any motion moves over: **dw**, **c$**, **y3j**, **dG**, **d/end**, **>}**. Doubled, they work on whole lines: **dd**, **cc**, **yy**, **>>**. Y is yy.

Other

ZZ	Write the file if it was changed, and quit.
ZQ	Quit without writing.
^G	Show the file's name, length, and where you are in it.
^L	Redraw the screen.

Command line

:w [*file*]	Write the file, or write to *file*.
:q, :q!	Quit; ! quits even if there are unsaved changes.
:wq, :x	Write and quit (:x writes only if there were changes).
:e[!] [*file*]	Edit another file, or the same file again.
:r *file*	Read *file* in below the current line.
:*n*, :$	Go to line *n*, or the last line.
:[*range*]s/*old*/*new*/[g]	Replace *old* with *new*, once in each line or everywhere (g).
:[*range*]d	Delete lines.
:help	A reminder of these.

A *range* is a line number, . (this line), $ (the last), two of them with a comma (2,10), or % (every line). Without one, a command works on the current line.

Options

-w *cols*	The terminal's width (default 80).
-h *rows*	The terminal's height (default 24).

Restrictions

Searches are for plain text, not regular expressions. There is one register, there are no marks, no visual mode, no :set, and no redo. The text must fit in memory: about 25K. Undo remembers as many changes as fit in its own space, and forgets the oldest first.

OS-9 text files end their lines with a carriage return. **vi** reads files that end them with CR, LF, or CR LF, and writes CR.

The arrow keys work in insert mode because **vi** sets bit 7 of $FF42 while it runs, so that the firmware sends them untranslated (see **tfr911h**(4)). If **vi** is killed, they stay that way until the next **vi** ends.

Examples

Change every foo to bar, write, and quit:

```
:%s/foo/bar/g
:wq
```

Delete from here to the line that contains end, then put it back:

```
d/end
u
```

See Also

unix(1), **tfr911h**(4)

Section 4: Special Files

This section describes the hardware the TFR/911H firmware presents to NitrOS-9: the addresses in the 6309's I/O page that a program can read and write, and what they do.

Name	Description
sn76489(4)	The sound chip
tfr911h(4)	The hardware the firmware emulates: memory, interrupts, console, disk, clock, keyboard

Name

sn76489 – the sound chip

Syntax

`$FF41`, write only

Description

The TFR/911H firmware emulates a Texas Instruments SN76489 sound generator at `$FF41`, the address of the one in the Color Computer's Game Master Cartridge. It has three square-wave tone voices and a noise voice, each with sixteen volumes. The firmware mixes them at 31,250 samples a second, with the keyclick and bell sounds, and plays the result as pulse-width-modulated audio on GPIO 8 (see Appendix A).

The chip is programmed one byte at a time. A byte with bit 7 set *latches* a voice and a register and gives the low four bits of the value; a byte with bit 7 clear gives the high six bits of a tone's period:

Byte	Meaning
`1 vv 0 dddd`	Voice `vv` (0–2): low 4 bits of its period
`0 0 dddddd`	The high 6 bits of the latched voice's period
`1 vv 1 aaaa`	Voice `vv` (0–3): attenuation, 2 dB a step; `1111` is off
`1 11 0 0 f rr`	The noise voice: `f`=1 white noise, 0 periodic; `rr` its rate, 0–2, or 3 to follow voice 2

A tone's period N (1–1023) gives a frequency of $3{,}579{,}545 / (32 \times N)$ Hz, so A440 is $N = 254$, and the lowest note is about 109 Hz.

The control register at `$FF42` is read and write:

bit 0 Keyclicks on (see **click**(1)).

bit 1 The bell sounds on `^G` (see **beep**(1)).

bit 7 Arrow keys untranslated (see **tfr911h**(4)).

Examples

In C, compiled with CMOC, from **tetris**(6):

```
#define SND (*(volatile unsigned char *)0xFF41)

void note(unsigned char voice, unsigned n) {   /* n = 3579545/32/Hz */
    SND = 0x80 | (voice << 5) | (n & 15);
    SND = (n >> 4) & 0x3F;
}

void volume(unsigned char voice, unsigned char att) { /* 15 is off */
    SND = 0x90 | (voice << 5) | att;
}
```

See Also

beep(1), **click**(1), **tfr911h**(4), Appendix A

Name

tfr911h – the hardware the TFR/911H firmware emulates for NitrOS-9 Level 2

Description

To NitrOS-9, the TFR/911H is a small Color Computer 3. The firmware's second core answers every bus cycle of the 6309: reads and writes of RAM go to the RP2350B's memory through the emulated MMU, and those of the I/O page, `$FF00–$FFFF`, go to the devices below. Locations not listed read back what was last written to them. The 6309 runs in 6809 mode.

Memory 256K of RAM, in 32 blocks of 8K. The GIME's MMU maps eight blocks into each of two tasks' 64K address spaces: `$FFA0–$FFA7` for task 0 and `$FFA8–$FFAF` for task 1. `$FF90` bit 6 enables the MMU and bit 5 the GIME's interrupts; `$FF91` bit 0 picks the task. `$FE00–$FEFF` is the constant page, the same in every map.

Interrupts `$FF92` bit 3 enables the vertical border (VBORD) interrupt, which a timer in the firmware raises 60 times a second; reading `$FF92` acknowledges it. The 6850's interrupt joins it on the 6309's IRQ line. There is no FIRQ.

Console An MC6850 ACIA at `$FF14` (status and control) and `$FF15` (data). Its transmitter is always ready; characters go to the tether at once.

Disk EmuDsk, a virtual hard disk whose sectors the tether reads and writes in a file:

`$FF80–$FF82`	The logical sector number, 24 bits.
`$FF83`	Write a command: 0 reads a sector, 1 writes one. Read the status: 0 if it worked.
`$FF84–$FF85`	The address of the 256-byte buffer.
`$FF86`	The drive number.

The bus waits while the tether moves the sector over USB, so the status is ready in the instruction after the command.

Clock `$FFC0–$FFC7` hold the century, year, month, day, day of the week, hour, minute, and second, in binary, in the format of Jeff Vavasour's CoCo 3 emulator. The tether sends your computer's local time when it connects, and the firmware keeps it going.

Sound `$FF41` and `$FF42`; see **sn76489**(4).

Info port The firmware's report on itself, for **info**(1). Write `$A5` to `$FF43`; when `$FF43` reads 1, each read of `$FF44` gives the report's next character, and 0 after the last. The report is lines of the form *label* `TAB` *value* `LF`.

Keyboard Keys typed in the tether reach OS-9 through the 6850. The tether turns Delete into Backspace and Enter into a carriage return. The arrow keys arrive as a Color Computer 3 sends them, unless bit 7 of `$FF42` is set:

Key	CoCo code	Bit 7 set	Note
Up	`$0C`	128	
Down	`$0A`	129	
Left	`$08`	130	The same as Backspace, as CoCo code
Right	`$09`	131	The same as Tab, as CoCo code
Page Up	—	132	Dropped, as CoCo code
Page Down	—	133	Dropped, as CoCo code

Files

`v4/tfr911h/v4_nitros9_l2.h`	The emulation.
`v4/tfr911h/v4_audio.h`	The sound chip and the mixer.

See Also

info(1), **sn76489**(4), **tether**(8)

Section 6: Games

This section describes the games. Both draw with ANSI escape sequences on the terminal the tether runs in.

Name	Description
tetris(6)	Tetris, with music
zm(6)	A Z-machine, for Zork and the other Infocom games

Name

tetris – Tetris, with music

Syntax

tetris

Description

Pieces fall into a well ten cells wide and twenty deep. Move and turn them so they make full rows, which disappear. The game draws in color on the terminal the tether runs in, shows the next piece and a ghost of where the falling one will land, and deals the seven pieces from a shuffled bag, so no piece is ever long in coming.

Left Right, `a d`	Move the piece.
Up, `w`	Turn it.
Down, `s`	Drop it a row.
Space	Drop it all the way.
`p`	Pause.
`m`	Turn the music off or on.
`q`	Quit.

Clearing one, two, three, or four rows at once scores 40, 100, 300, or 1,200 points times the level plus one. Dropping a piece scores a point a row, or two with Space. Every ten rows is a new level, and the pieces fall faster, up to level 19.

With a speaker connected, the game plays *Korobeiniki* on the sound chip, faster as the levels go up, with sounds for moving, turning, landing, clearing rows, a new level, and the end of the game.

Restrictions

It needs an ANSI color terminal of at least 80 by 24. It turns keyclicks off while it runs.

See Also

click(1), **sn76489**(4)

Name

zm – play Infocom's games, and others written for the Z-machine

Syntax

zm [`-w` *cols*] [`-h` *rows*] [`-m` *kbytes*] [`-d`] *story*

Description

Infocom wrote its games for an imaginary computer, the Z-machine, and sold them with an interpreter for each real one. **zm** is a Z-machine interpreter for OS-9. It plays *version 3* story files: most of Infocom's games from 1980 to 1987 (Zork I–III, The Hitchhiker's Guide to the Galaxy, Planetfall, Enchanter, Deadline, and many more), and games written with Inform and compiled with `-v3`.

zm looks for *story*, then *story*`.z3`, and, if the name has no directory in it, the same in `/DD/GAMES`. So **`zm zork1`** finds `/DD/GAMES/zork1.z3`.

The top line of the screen is the status line: where you are, and your score and moves, or the time in the games that keep it. The text scrolls beneath it, word-wrapped to the screen's width, and pauses with `[MORE]` when a screen goes by without your typing. Games that split the screen, as Seastalker does, can.

The game's commands `save` and `restore` ask for a file name, offering the story's name with `.sav`; save files go in the data directory. `restart`, `verify`, and `quit` work as in the games' manuals.

The part of the story the game can change, its *dynamic memory*, is kept in RAM. So is as much of the rest as fits—about 40K, got by growing the process's data area with `F$Mem`. If the story is bigger than that, the rest is read from the file in 512-byte pages as the game needs them, and the least recently used page gives way. Panic, which is 37K, is held in memory whole; Zork I, at 87K, is paged, and still answers a command in under a second.

Options

`-w` *cols*	The terminal's width (default 80).
`-h` *rows*	The terminal's height (default 24).
`-m` *kbytes*	Use at most this much memory for the story (8–60).
`-d`	A dumb terminal: no status line, no escape sequences, no `[MORE]`.

Restrictions

Only version 3 stories are played; **zm** says which version a story is if it is another. There is no transcript (`script`), and there are no sounds but the two beeps of The Lurking Horror, which it plays on the sound chip.

Diagnostics

If the story does something impossible, **zm** stops with a message such as:

```
*** zm: stack underflow (PC $1A2B3) ***
```

Examples

```
OS9:zm zork1
ZORK I: The Great Underground Empire
...
West of House
You are standing in an open field west of a white house, with a boarded
front door.
There is a small mailbox here.

>open mailbox
```

To put story files on the disk, see **nitros9-l2**(8).

Files

`/DD/GAMES/*.z3`	Story files.
story`.sav`	Saved games.

See Also

nitros9-l2(8), Appendix B

Section 8: Maintenance

This section describes the programs that run on the computer the TFR/911H is plugged into, the firmware's console, and how to build the system and keep a disk up to date.

Name	Description
nitros9-l2(8)	Build NitrOS-9 Level 2 for the TFR/911H, and keep a disk up to date
tcl(8)	The firmware's console
tether(8)	The board's terminal, disk, and clock

Name **nitros9-l2** – build NitrOS-9 Level 2 for the TFR/911H, and keep a disk up to date

Syntax **make** [*target*] [`DISK=`*image*] [`STORIES=`*files*] [`OS9_TOOL=`*path*]

Description The `v4/nitros9-l2` directory builds NitrOS-9 Level 2 for the TFR/911H from a NitrOS-9 source tree, with the commands described in this manual, and makes a disk image. It needs **lwasm** from LWTOOLS, **os9** from ToolShed, **tclsh**, **python3**, and, for the C programs, CMOC.

`make`	Build the kernel, `OS9Boot`, the commands, and `build/tfr911-level2.dsk`, and the firmware's copy of the kernel track, `../tfr911h/nitros9_l2_rom.h`.
`make install`	Bring the disk *DISK* (default `level2.dsk`) up to date: a new `OS9Boot`, and every file in `CMDS` and `SYS`. Other files are left alone.
`make stories`	Copy the story files named by *STORIES* to `/DD/GAMES` on *DISK*.
`make build/zm-host`	Build **zm** for this computer, for testing; also `build/vi-host`.
`make clean`	Remove the build.

The firmware is built from `v4`: **`make build/tfr911h_v4.uf2`**, then loaded with **picotool**.

Restrictions Quit the tether before changing the disk it is serving, and reset the board after `make install` so that it boots the new `OS9Boot`.

Examples

```
$ cd v4/nitros9-l2
$ make OS9_TOOL=~/toolshed/build/unix/os9/os9
$ make install DISK=level2.dsk
$ make stories STORIES="~/if/zork1.z3 ~/if/hhgg.z3" DISK=level2.dsk
```

Files

`rmake-l2.tcl`	Which modules go in `OS9Boot` and on the disk.
`unix.tcl`	The Unix-style aliases, and changes to the Shell and **grep**.
`modules/`	The TFR/911H's own modules: the clock, **clear**, **beep**, **click**.
`games/`	The C programs: **tetris**, **zm**, **vi**, **info**.
`helpmsg.unix`	The **man** pages for what was added.
`doc/manual.tex`	This manual.

See Also **tcl**(8), **tether**(8)

Name **tcl** – the TFR/911H firmware's console

Syntax `TCL>` *command*

Description When the TFR/911H starts, its firmware waits at the `TCL>` prompt, a Tcl interpreter that runs on the RP2350B. Besides the Tcl language, it has these commands:

l2	Boot NitrOS-9 Level 2 from EmuDsk drive 0.
bye	Boot TurbOS, the OS-9 Level 1 system in the firmware.
beep [*hz* [*ms*]]	Play a tone through the speaker (default 880 Hz for 200 ms), to test the wiring.
click [`on`|`off`]	Turn keyclicks on or off, and say which.

Examples

```
TCL> beep 440 1000
TCL> l2
```

See Also **click**(1), **tether**(8)

Name

tether, **run-l2.sh** – the board's terminal, disk, and clock

Syntax

sh run-l2.sh
tether `-wire` *device* [`-disks` *images*] [`-boot` *command*] [`-pass_ctrl_c`] [*other options*]

Description

The tether is a program on your computer that talks to the TFR/911H over USB. It is the board's terminal: what you type goes to the 6309, and what the 6309 prints appears in the window. It is the board's disk: it reads and writes the sectors of disk image files for EmuDsk. And it sets the board's clock when it connects.

`v4/run-l2.sh` builds the tether and runs it with the options NitrOS-9 Level 2 needs. If NitrOS-9 is already running on the board, it simply reconnects.

`WIRE`	The board's USB serial device. On macOS, the only `/dev/cu.usbmodem*` device, if there is just one.
`DISK`	The disk image for drive 0. The default, `nitros9-l2/level2.dsk`, is copied from the build the first time.

Options

`-wire` *device*	The serial device.
`-disks` *images*	Disk image files, comma separated, for drives 0, 1, …
`-boot` *command*	A command to type at the `TCL>` prompt, such as `l2`. Ignored if the 6309 is already running.
`-pass_ctrl_c`	Send `^C` (and `^\` and `^Z`) to the 6309, and quit the tether with `^]` instead.
`-level` *n*	The NitrOS-9 level, for the tether's diagnostics.

Restrictions

Only one program can have the board's serial device open. Quit the tether before you flash the firmware or change a disk image it is serving.

Files

`v4/_log`	The tether's diagnostics, including the firmware's statistics every ten seconds.

See Also

nitros9-l2(8), **tcl**(8), **tfr911h**(4)

Connecting a Speaker A

The TFR/911H makes sound as pulse-width-modulated audio on GPIO 8 of the RP2350B: an 8-bit level carried at about 977 kHz, far above hearing. A filter and a small amplifier turn it into sound. The firmware was written for an Adafruit PAM8302, a 2.5 W class D amplifier on a small board.

A.1 Wiring

GPIO 8 is pin 1 of the 2-by-10 header J3, and is not connected to the 6309. Wire the PAM8302 like this:

J3 pin	Signal	PAM8302
2	VBUS (5 V)	Vin
6	GND	GND, and A–
1	GPIO 8	A+, through a 1 kΩ resistor, with a 10 nF capacitor from A+ to GND

The resistor and capacitor make a low-pass filter that takes out the carrier. Connect the speaker to the PAM8302's + and – terminals, and turn its volume control down before you start.

A.2 Trying It

At the `TCL>` prompt, before an OS is booted:

```
TCL> beep 440 1000
```

should play an A for a second. Then, in NitrOS-9, **beep** rings the bell, every key clicks (**click off** stops that), and **tetris** plays music.

A.3 How It Works

A timer interrupt on the RP2350B's first core runs 31,250 times a second. Each time, it takes any bytes the 6309 has written to the SN76489, works out the next sample of the three tone voices, the noise voice, and the keyclick or bell, adds them, and sets the PWM level. The 6309's writes reach it through a queue, so the core that runs the bus never waits.

How the Software Was Tested B

The C programs were written so that the same source builds for OS-9 with CMOC and for a Unix computer with an ordinary C compiler. That let each one be tested against a reference on the big computer, and then on the board.

B.1 zm

- CZECH, the Comprehensive Z-machine Emulation Checker, compiled for version 3 with Inform: all 349 of its tests pass, on the Unix build and on the TFR/911H's 6309, where the run takes about five seconds. They pass too with memory cut to 8K, which makes nearly every read a page from the file.
- Zork I, II, and III and Inform's Adventure: transcripts of the same commands give the same text, word for word, as **dfrotz**, a widely used interpreter, with the story wholly in memory and with only 16K to 40K for it.
- On the board: Zork I answers each command in 0.4 to 0.9 seconds; `save`, `restore`, `restart` work.

B.2 vi

- 123 editing commands—motions, operators, counts, undo, `.`, searches, `:s` and ranges—were typed into Vim and into **vi**, on a pseudo-terminal at the same pace, and the files they wrote compared: all the same.
- 27 sessions on a 300-line file with tabs and long lines checked that what **vi** shows is what is in the file, after scrolling, sideways scrolling, and edits.
- On the board: moving and typing answer in under 0.05 seconds, scrolling in 0.15, and replacing 178 strings on 116 lines in 1.8.

B.3 info

Its report was compared with what NitrOS-9's own **procs**, **mdir**, **mfree**, and **free** say, on the board, and with a program running in the background.

B.4 The Firmware

Each change was tried on the board: NitrOS-9 boots, the disk and clock work, Tetris's writes reach the sound chip (182 of them in one game, by the firmware's count), and the arrow keys arrive as they should with bit 7 of `$FF42` clear and set.